

Studentessa: Finiguerra Alessia

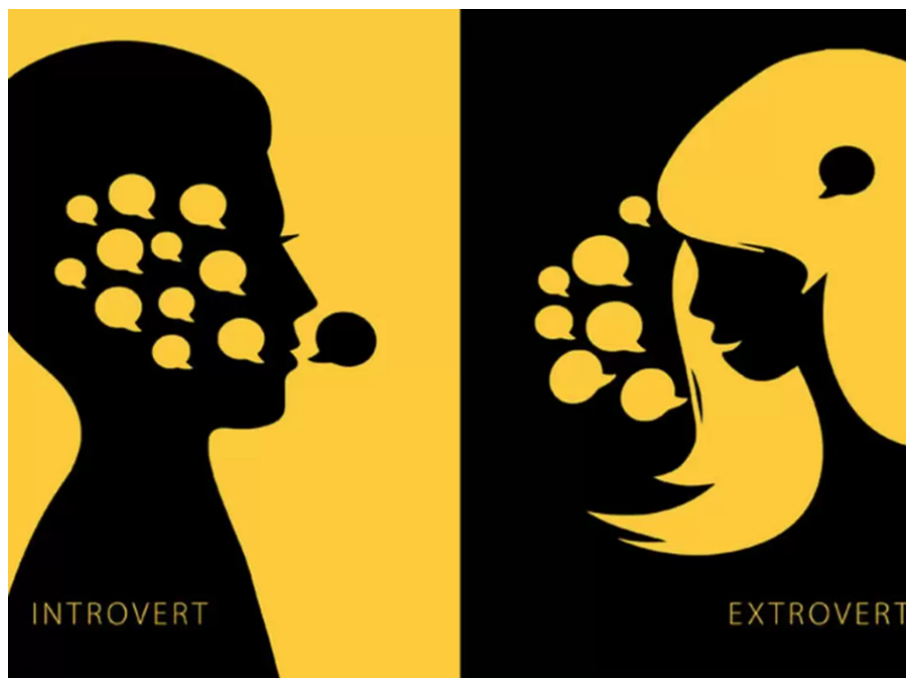
Matricola: 735326

Email: a.finiguerra1@studenti.uniba.it

URL GitHub:

<https://github.com/alefiniguerra01/>

ICON-Personality_Prediction.git



***SISTEMA DI PREDIZIONE DELLA
PERSONALITA':
INTROVERSO VS ESTROVERSO***

A.A. 2024/2025

INDICE

1	INTRODUZIONE	1
1.1	ARGOMENTI DI INTERESSE	1
2	APPRENDIMENTO SUPERVISIONATO	1
2.1	STRUMENTI UTILIZZATI	1
2.2	DATA EXPLORATION E DATA PREPROCESSING	2
2.2.1	DATASET	2
2.2.2	VALORI DUPLICATI	2
2.2.3	EDA: EXPLORATORY DATA ANALYSIS	3
2.2.4	OUTLIER	4
2.2.5	VALORI NULLI	4
2.2.6	CONVERSIONE DELLE FEATURES CATEGORICHE	5
2.3	MODEL TRAINING AND EVALUATION	5
2.4	K-NEAREST NEIGHBORS	7
2.5	RANDOM FOREST	9
2.6	DECISION TREE	11
3	SISTEMA ESPERTO	13
3.1	STRUMENTI UTILIZZATI	14
3.2	KNOWLEDGE BASE	14
3.3	INTERFACCIA UTENTE	16

1 INTRODUZIONE

La personalità di una persona è l'insieme delle caratteristiche psicologiche che influenzano il modo in cui pensa, sente e si comporta nelle diverse situazioni. Essa ci rende unici e può comprendere tratti come l'emotività, la socievolezza, la stabilità e la creatività.

Tra le molte dimensioni della personalità, una classificazione degli individui può essere quella tra persone “*introverse*” e persone “*estroverse*”: le persone introversive tendono a essere più riservate, riflessive e a preferire ambienti tranquilli, mentre quelle estroverse sono generalmente più socievoli, energiche e stimolate dal contatto con gli altri.

L'obiettivo di questo progetto è quello di costruire un modello in grado di prevedere accuratamente la personalità di un individuo, utilizzando le principali tecniche di apprendimento automatico.

1.1 ARGOMENTI DI INTERESSE

Gli argomenti affrontati in questo progetto sono:

- **Apprendimento supervisionato:** è una tecnica in cui il modello impara da un dataset fornito in input; attraverso l'applicazione di vari algoritmi, il modello viene poi addestrato per essere in grado di effettuare delle previsioni accurate;
- **Apprendimento supervisionato con iperparametri:** attraverso la definizione di iperparametri si affinano le capacità del modello in modo da massimizzare l'accuratezza delle sue previsioni;
- **Sistema esperto:** viene creata una base di conoscenza e si applica un motore inferenziale che sarà in grado di ragionare logicamente e arrivare ad una conclusione, emulando le capacità decisionali di un esperto umano.

2 APPRENDIMENTO SUPERVISIONATO

2.1 STRUMENTI UTILIZZATI

Il progetto è stato realizzato nel linguaggio *Python* utilizzando l'IDE *Visual Studio Code* come ambiente di sviluppo. Tra le varie librerie necessarie abbiamo:

- **Pandas:** per la manipolazione e l'analisi dei dati;

-
- **Matplotlib e Seaborn:** per creare e gestire i grafici;
 - **Scikit-learn:** per addestrare e valutare il modello;
 - **Numpy:** per eseguire operazioni matematiche su specifici dati.

2.2 DATA EXPLORATION E DATA PREPROCESSING

La fase di *Data Exploration* è la fase iniziale di ogni progetto di Machine Learning. Il suo obiettivo principale è quello di esplorare il dataset per comprenderne le sue caratteristiche.

2.2.1 DATASET

Per valutare e fare previsioni, un modello di Machine Learning ha bisogno di un set di dati da cui “imparare”, che farà quindi da base per costruire la sua conoscenza.

Per predire il tipo di personalità, il mio modello ha utilizzato il dataset “*personality_dataset.csv*” contenente 2900 righe e 8 colonne.

Di seguito abbiamo la spiegazione delle varie features:

- **Time_spent_Alone:** numero di ore che l’individuo trascorre da solo ogni giorno;
- **Stage_fear:** presenza della paura da palcoscenico;
- **Social_event_attendance:** frequenza di partecipazione ad eventi sociali;
- **Going_outside:** frequenza con cui l’individuo esce;
- **Drained_after_socializing:** presenza della sensazione di svuotamento dopo aver socializzato;
- **Friends_circle_size:** numero degli amici più intimi;
- **Post_frequency:** frequenza di pubblicazione sui social media;
- **Personality:** questa rappresenta la **variabile target** e indica appunto se l’individuo è “Introverso” o “Estroverso”.

2.2.2 VALORI DUPLICATI

Per garantire l’integrità dei dati e prevenire un addestramento distorto del modello, è stato ripulito il dataset eliminando eventuali valori duplicati presenti al suo interno, ottenendo un dataset composto di circa 2500 righe e 8 colonne.

2.2.3 EDA: EXPLORATORY DATA ANALYSIS

L'EDA (*Exploratory Data Analysis*) è la fase iniziale dell'analisi dei dati in cui si esplorano, visualizzano e comprendono le caratteristiche principali di un dataset.

L'obiettivo è identificare anomalie, distribuzioni e relazioni tra variabili, senza ancora applicare modelli predittivi.

Per visualizzare la frequenza con cui determinati valori compaiono nel dataset, sono stati creati dei grafici per confrontare visivamente come essi sono distribuiti rispetto alle relative caratteristiche.

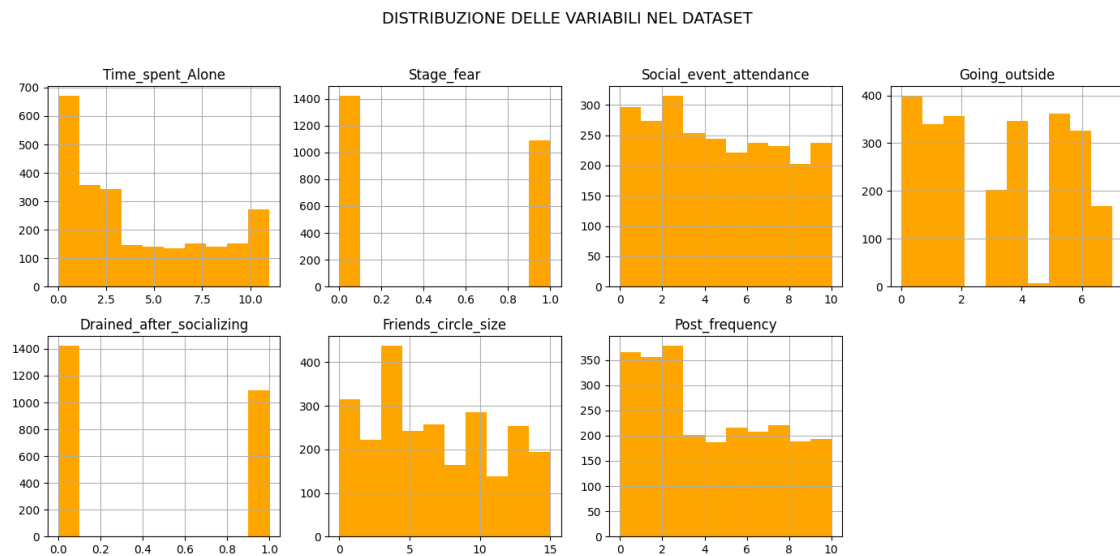


Figure 1: *Distribuzione delle variabili*

Successivamente, è stata effettuata l'analisi della correlazione tra le varie features con la feature target "*Personality*" per comprendere meglio quali variabili influiscono maggiormente sulla variabile target.

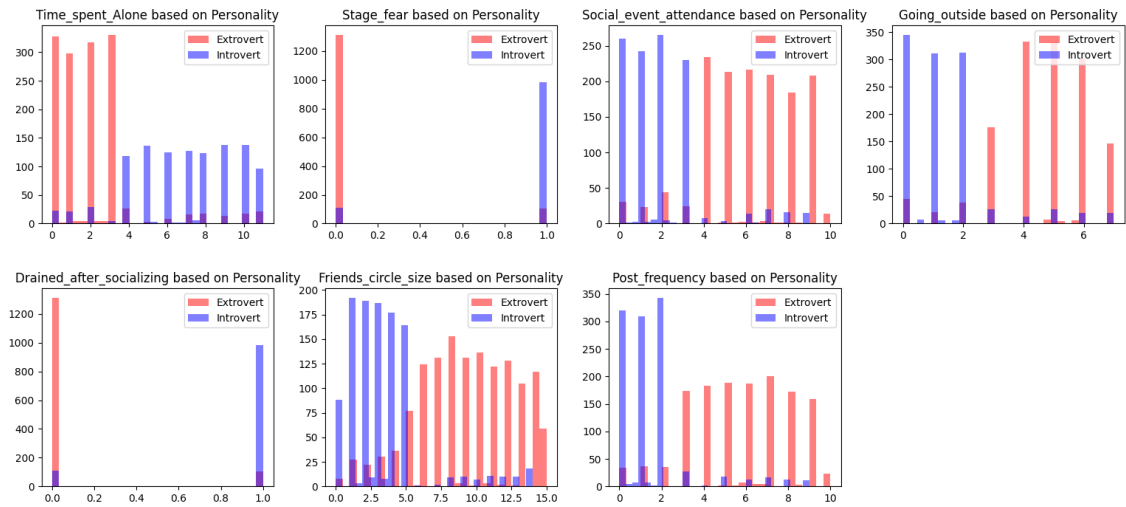


Figure 2: *Incidenza delle variabili*

Come si può notare dai grafici, l'analisi esplorativa conferma che le features *Drained_after_socializing*, *Stage_fear*, e, in misura minore, *Time_spent_Alone* e *Social_event_attendance* sono quelle che contribuiscono maggiormente nella distinzione tra introversi ed estroversi.

2.2.4 OUTLIER

All'interno di un dataset possono essere presenti dei valori che a primo impatto possono sembrare anomali, i cosiddetti “*outlier*”, ovvero dei valori che si discostano in modo significativo dal resto delle osservazioni.

Dopo un'attenta analisi del dataset, si è deciso di non procedere con l'eliminazione degli outlier identificati in quanto si è supposto che tali valori non rappresentino errori di misurazione o di inserimento ma piuttosto variazioni autentiche e naturali del comportamento umano.

2.2.5 VALORI NULLI

Nel dataset a disposizione erano presenti dei valori nulli sia per le features numeriche sia per quelle categoriche.

Per le features numeriche, si è deciso di sostituire i valori mancanti utilizzando un approccio basato sull'algoritmo KNN: per ogni valore mancante, sono stati trovati i tre vicini che avevano il valore più simile ad esso; l'algoritmo KNN ha calcolato poi la media dei tre valori e l'ha usata per sostituire il valore mancante.

Quest'approccio è stato valutato come il più performante perché tiene conto dei valori delle altre features e delle relazioni che intercorrono tra esse, portando a un dataset più realistico.

Per le features categoriche, invece, si è deciso di riempire i valori mancanti con la moda di ogni specifica feature.

2.2.6 CONVERSIONE DELLE FEATURES CATEGORICHE

Gli algoritmi di apprendimento supervisionato richiedono che le features di input siano rappresentate mediante valori numerici.

Nel nostro dataset sono presenti due features categoriche (oltre alla variabile target). Per questo motivo, si è reso necessario convertire i loro valori in valori numerici, rappresentando il valore “Yes” come 1 e il valore “No” come 0.

Per la variabile target, invece, si è utilizzato un *LabelEncoder* che, dopo aver analizzato la feature “Personality”, ha assegnato un numero per ogni valore trovato, convertendo alla fine “Extrovert” con 0 e “Introvert” con 1.

2.3 MODEL TRAINING AND EVALUATION

Terminata la fase di *Preprocessing*, si procede con l'addestramento del modello.

Per eseguire la predizione sulla personalità di un individuo, sono stati utilizzati diversi modelli di classificazione. Tenendo conto della quantità limitata dei dati presenti nel dataset, si è preferito scegliere modelli con complessità più semplice rispetto alla regressione lineare o alle reti neurali che richiedono una miglior precisione legata ad un dataset molto più fornito.

I modelli di classificazione utilizzati sono: **Random Forest**, **Gradient Boosting**, **Decision Tree**, **SVM**, e **KNN**.

All'interno del file “*train_eval.py*” sono stati addestrati e testati i diversi modelli senza l'utilizzo di particolari parametri, solo per prendere visione dei risultati iniziali.

Per l'occasione, si è deciso di considerare l'80% dei dati del dataset come training set e il restante 20% come test set.

Sono state condotte valutazioni preliminari delle prestazioni dei modelli, includendo metriche chiave come *accuracy*, *precision* e *recall* all'interno di un *classification report*.

Di seguito i risultati iniziali:

```

----- Decision Tree -----
Accuracy: 0.849
Classification Report:

```

	precision	recall	f1-score	support
Extrovert	0.86	0.87	0.87	282
Introvert	0.83	0.82	0.83	221
accuracy			0.85	503
macro avg	0.85	0.85	0.85	503
weighted avg	0.85	0.85	0.85	503

Figure 3: *Decision Tree Classification Report*

```

----- Gradient Boosting -----
Accuracy: 0.932
Classification Report:

```

	precision	recall	f1-score	support
Extrovert	0.94	0.94	0.94	282
Introvert	0.92	0.92	0.92	221
accuracy			0.93	503
macro avg	0.93	0.93	0.93	503
weighted avg	0.93	0.93	0.93	503

Figure 4: *Gradient Boosting Classification Report*

```

----- KNN -----
Accuracy: 0.920
Classification Report:

```

	precision	recall	f1-score	support
Extrovert	0.92	0.94	0.93	282
Introvert	0.92	0.90	0.91	221
accuracy			0.92	503
macro avg	0.92	0.92	0.92	503
weighted avg	0.92	0.92	0.92	503

Figure 5: *KNN Classification Report*

```

----- Random Forest -----
Accuracy: 0.905
Classification Report:

```

	precision	recall	f1-score	support
Extrovert	0.91	0.92	0.92	282
Introvert	0.89	0.89	0.89	221
accuracy			0.90	503
macro avg	0.90	0.90	0.90	503
weighted avg	0.90	0.90	0.90	503

Figure 6: *Random Forest Classification Report*

```

----- SVM -----
Accuracy: 0.932
Classification Report:

```

	precision	recall	f1-score	support
Extrovert	0.94	0.94	0.94	282
Introvert	0.92	0.92	0.92	221
accuracy			0.93	503
macro avg	0.93	0.93	0.93	503
weighted avg	0.93	0.93	0.93	503

Figure 7: *SVM Classification Report*

```

----- Riepilogo Accuracy -----

```

	Modello	Accuracy
1	Gradient Boosting	0.932406
2	SVM	0.932406
3	KNN	0.920477
4	Random Forest	0.904573
5	Decision Tree	0.848907

Figure 8: *Riepilogo Accuracy*

Dalla prima analisi e dalla figura riportante i risultati delle *accuracy* di ogni modello valutato, è possibile notare che il *Gradient Boosting* e l'*SVM* sono risultati essere i modelli più performanti, con identici e alti livelli di *accuracy* pari al 93.2% circa.

Seguono poi il *KNN* e il *Random Forest* con valori di *accuracy* leggermente inferiori, rispettivamente del 92% circa e del 90.4% circa.

Infine, il *Decision Tree* si è rivelato essere il modello con le prestazioni più basse, con un livello di *accuracy* pari al 84.8% circa.

Per migliorare le prestazioni degli ultimi tre modelli, è stata condotta un'analisi più approfondita per identificare i parametri ottimali dei modelli **KNN**, **Random Forest** e **Decision Tree**, essendo quelli con *accuracy* più basse.

2.4 K-NEAREST NEIGHBORS

Il *K-Nearest Neighbors (KNN)* è un algoritmo il cui funzionamento si fonda sul principio di "somiglianza": per classificare un nuovo dato, l'algoritmo individua i "*k*" punti più vicini a esso nel set di addestramento e assegna la classe che risulta essere più comune tra questi vicini.

Per aumentare la performance del modello, si è preso in considerazione il suo iperparametro principale, **n_neighbors**, che definisce il numero di vicini da consultare per la classificazione: un suo valore troppo basso può portare il modello a sovradattare i dati (*overfitting*), mentre un valore troppo alto rischia di rendere le decisioni troppo generiche (*underfitting*).

Per trovare il miglior valore per l'iperparametro *n_neighbors* che offrisse le migliori prestazioni, sono stati effettuati **10 run diversi**, ciascuno con una diversa suddivisione casuale dei dati in training set e test set: in ogni run, sono stati testati **30 valori diversi** (da 1 a 30), utilizzando una **Cross Validation a 5 fold** per stimare in modo più stabile e affidabile l'accuratezza del modello, riducendo la dipendenza da una singola suddivisione dei dati.

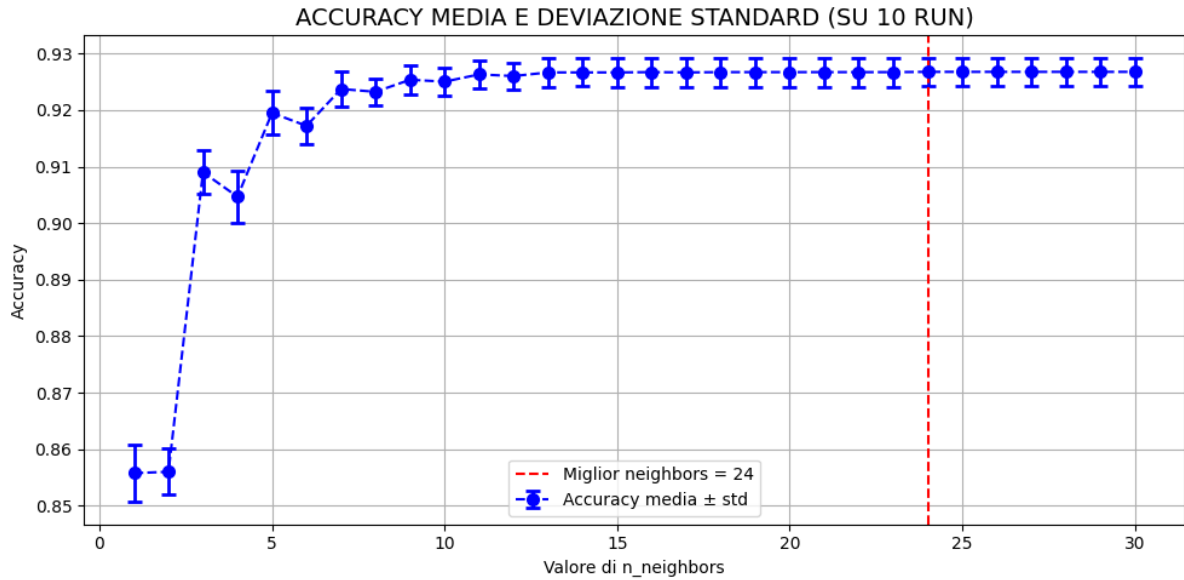


Figure 9: *Accuracy Media e Deviazione Standard per i valori di n_neighbors*

Attraverso il calcolo dell'*accuracy media* e della relativa *deviazione standard* per ogni valore di *n_neighbors* (aggregati per run) è stato generato il grafico soprastante che evidenzia l'andamento dell'accuratezza media al variare proprio del valore di *n_neighbors*: per valori bassi da 1 a 4, l'accuracy del modello è troppo bassa e la relativa deviazione standard troppo alta, sintomo di overfitting dei dati; dal valore 5, l'accuracy media cresce e le deviazioni standard sono contenute, fino a stabilizzarsi al valore 24, che indica quel valore che permette al modello di ottenere la combinazione ideale tra precisione, stabilità e, soprattutto, affidabilità, con un accuracy pari a 0.9268 e una deviazione standard pari a 0.0025.

Tabella riassuntiva delle run:

Run	CV Accuracy	CV Std
1	0.9193	0.0185
2	0.9205	0.0167
3	0.9199	0.0186
4	0.9213	0.0175
5	0.9248	0.0177
6	0.9186	0.0187
7	0.9184	0.0182
8	0.9229	0.0172
9	0.9161	0.0173
10	0.9159	0.0185

Figure 10: *Tabella riassuntiva delle run*

La tabella riassuntiva soprastante fornisce una panoramica delle prestazioni ottenute nelle 10 esecuzioni del modello KNN: per ogni run è stata calcolata la media delle accuratezze ottenute su tutti i valori di `n.neighbors` e la relativa deviazione standard.

I risultati evidenziano una **notevole stabilità** del modello: le **accuratezze medie** oscillano in un range molto ristretto (**da 0.9159 a 0.9248**), suggerendo che, indipendentemente dalla suddivisione dei dati, il classificatore mantiene prestazioni elevate e costanti. Anche le **deviazioni standard** restano sempre intorno a un valore basso (**0.017-0.019**) che indicano una limitata sensibilità all'iperparametro testato.

Per visualizzare graficamente l'andamento dei risultati delle 10 run, è stato creato un grafico per definire i valori ottenuti tramite cross-validation per le rispettive medie e deviazioni standard.

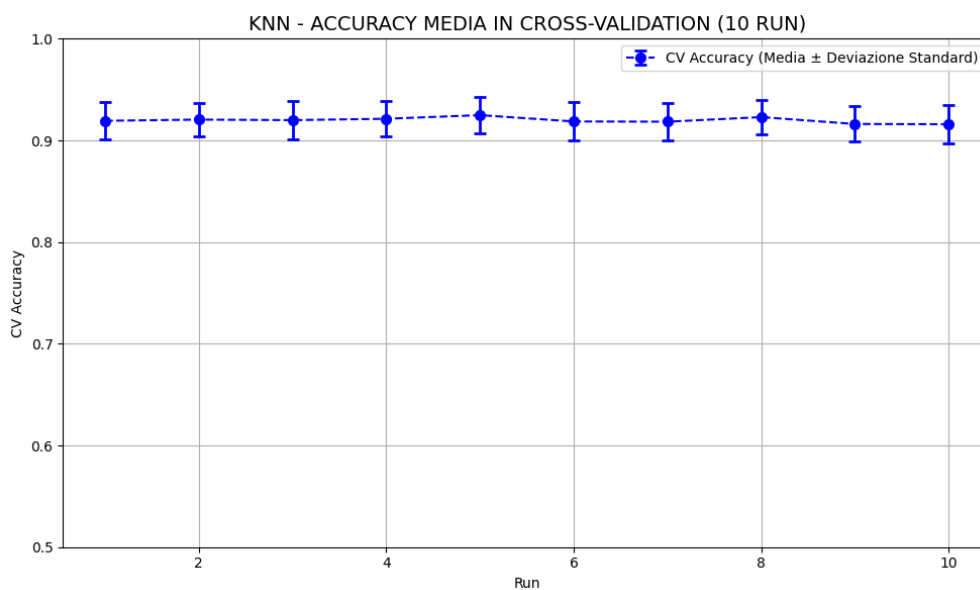


Figure 11: *KNN: Cross-Validation Accuracy per run*

2.5 RANDOM FOREST

Il *Random Forest* è un algoritmo di apprendimento supervisionato che costruisce una moltitudine di *alberi decisionali* durante la fase di training e aggrega i loro risultati per produrre una previsione finale. Questo approccio è noto per la sua robustezza e la sua capacità di ridurre l'*overfitting* rispetto a un singolo albero decisionale.

Anche per questo modello, si è scelto di adottare una metodologia basata sull'esecuzione di **10 run indipendenti**, ciascuna con una diversa suddivisione casuale del dataset in *training* e *test set* (con stratificazione sulla *variabile target*).

La ricerca degli iperparametri è stata eseguita mediante **RandomizedSearchCV** con 15 combinazioni casuali tra i valori proposti e con **Cross-Validation a 5 fold**.

L'elenco degli iperparametri testati è il seguente:

- **n_estimators**: numero di alberi nella foresta, campionato casualmente tra 50 e 200;
- **max_depth**: profondità massima dell'albero, campionato tra i valori *None*, 5, 10, 15;
- **min_samples_split**: numero minimo di campioni per suddividere un nodo, scelto tra i valori 2, 5, 10;
- **min_samples_leaf**: numero minimo di campioni in una foglia, scelto tra i valori 1, 2, 4.

Tabella riassuntiva delle run:			
Run	CV Accuracy	CV Std	
1	0.9268	0.0138	
2	0.9273	0.0144	
3	0.9273	0.0161	
4	0.9278	0.0146	
5	0.9313	0.0130	
6	0.9258	0.0095	
7	0.9253	0.0109	
8	0.9298	0.0144	
9	0.9228	0.0091	
10	0.9233	0.0062	

Figure 12: *Tabella riassuntiva delle run*

Per ogni run, sono stati registrati la media e la deviazione standard dell'accuracy ottenuta in Cross-Validation per la miglior combinazione di iperparametri trovata.

La tabella soprastante mostra un'**elevata stabilità** del modello tra le diverse esecuzioni: l'**accuracy media** in Cross-Validation si è mantenuta costantemente in un range compreso tra **0.9228 e 0.9313**, con **deviazioni standard** generalmente **inferiori a 0.014**, a indicare una buona stabilità e generalizzazione del modello.

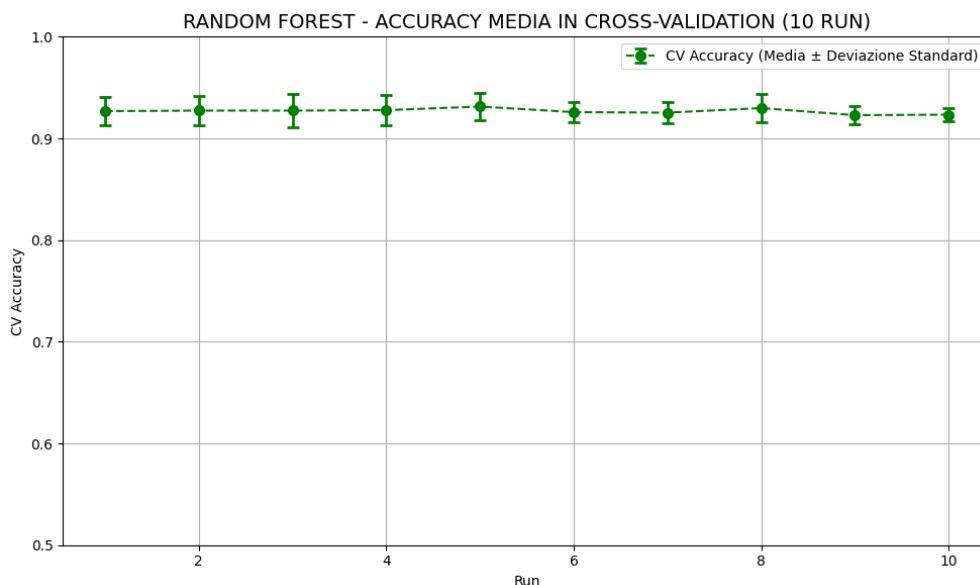


Figure 13: *Random Forest: Cross-Validation Accuracy per run*

L'andamento dei risultati delle 10 run è stato visualizzato anche graficamente mediante un grafico a barre con errore, evidenziando che le performance sono risultate coerenti nonostante delle lievi oscillazioni tra i valori delle deviazioni standard registrati lungo le run, che indicano che il modello potrebbe comunque essere sensibile ad alcune configurazioni del dataset.

Si può concludere dicendo che il *Random Forest* è un modello affidabile anche in presenza di variabilità nei dati di input.

2.6 DECISION TREE

Il *Decision Tree* è un algoritmo di apprendimento supervisionato che opera creando una struttura a flusso, simile ad un albero, dove ogni nodo interno decisionale rappresenta una "domanda" su una specifica feature dei dati e ogni ramo una *risposta*. Seguendo il percorso dall'alto verso il basso, si arriva a una "foglia" finale che contiene la previsione della classe.

Per ottimizzare le prestazioni di questo modello, è stato adottato un approccio simile a quello utilizzato per Random Forest, con una strategia basata su **10 run indipendenti**, ciascuna con una suddivisione casuale del dataset, per ottenere valutazioni più affidabili e stabili rispetto a una singola run.

Anche in questo caso, per ogni run, il dataset è stato suddiviso con stratificazione della variabile target e si è adottata una **RandomizedSearchCV**, ovvero una ricerca randomica di combinazioni di iperparametri, valutate mediante **Cross-Validation a 5 fold**.

La procedura di tuning ha coinvolto tre iperparametri principali:

- **max_depth**: profondità massima dell'albero (per controllare l'overfitting), scelto tra i valori *None, 3, 5, 10, 15, 20*;
- **min_samples_split**: numero minimo di campioni per suddividere un nodo, scelto tra i valori *2, 5, 10, 15*;
- **min_samples_leaf**: numero minimo di campioni in una foglia, scelto tra i valori *1, 2, 4, 6*.

In ciascuna run, sono state valutate 15 combinazioni casuali di iperparametri, con l'obiettivo di individuare quella che garantiva la migliore accuratezza media in Cross-Validation.



The image shows a terminal window with a dark background and light-colored text. At the top, the title 'Tabella riassuntiva delle run:' is displayed. Below it is a table with three columns: 'Run', 'CV Accuracy', and 'CV Std'. There are 10 rows of data, numbered 1 to 10. The 'CV Accuracy' values range from 0.9218 to 0.9313, and the 'CV Std' values range from 0.0062 to 0.0161.

Run	CV Accuracy	CV Std
1	0.9268	0.0138
2	0.9273	0.0144
3	0.9273	0.0161
4	0.9273	0.0151
5	0.9313	0.0130
6	0.9253	0.0089
7	0.9253	0.0109
8	0.9298	0.0144
9	0.9218	0.0079
10	0.9233	0.0062

Figure 14: *Tabella riassuntiva delle run*

I risultati sono stati sintetizzati in una tabella riassuntiva che raccoglie, per ciascuna delle 10 run, l'accuracy media ottenuta in Cross-Validation e la relativa deviazione standard.

Dalla tabella si può concludere che l'**accuracy media** in Cross-Validation si mantiene su **valori buoni e stabili** lungo le 10 run (**tra 0.9218 e 0.9313**), a testimonianza della robustezza del modello Decision Tree, soprattutto se correttamente ottimizzato.

I valori delle **deviazioni standard**, invece, si sono tenuti ad un livello mediamente **inferiore a 0.14**.

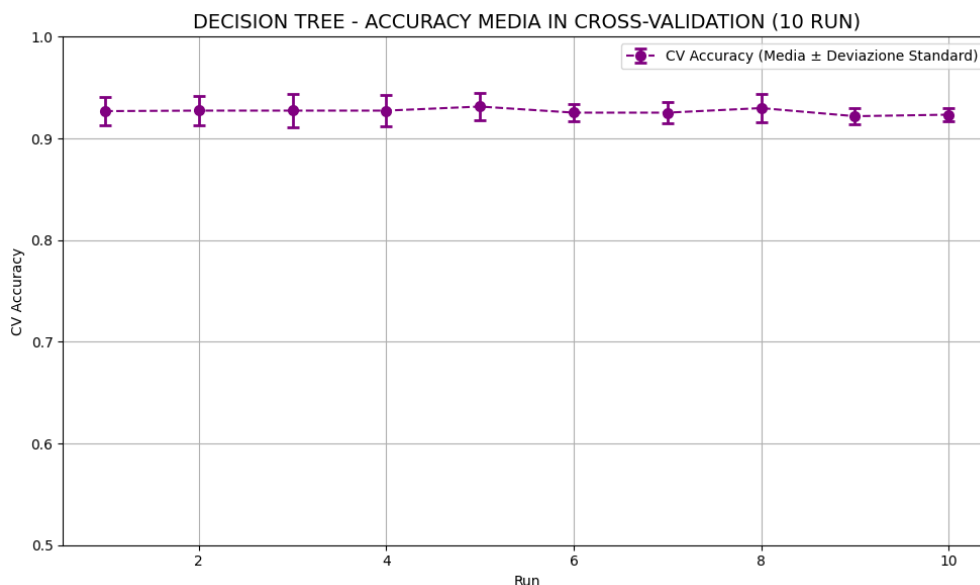


Figure 15: *Decision Tree: Cross-Validation Accuracy per run*

Il grafico soprastante visualizza graficamente i risultati del tuning effettuato sul modello selezionato, dimostrando che anche il *Decision Tree*, se opportunamente ottimizzato, può portare ad alti livelli di accuratezza nonostante una lieve variabilità dei valori delle deviazioni standard tra le run, che ci suggerisce che il modello potrebbe comunque essere sensibile ad alcune configurazioni del dataset.

3 SISTEMA ESPERTO

Un *sistema esperto* è un programma di intelligenza artificiale progettato per replicare le capacità di ragionamento di un esperto umano in un dominio specifico e ristretto.

È composto principalmente da tre parti:

- la **Base di Conoscenza (Knowledge Base)**, che contiene i dati e le regole del dominio;
- il **Motore Inferenziale (Inference Engine)**, ovvero il software che applica le regole ai dati per trarre conclusioni logiche;
- l'**Interfaccia Utente (User Interface)**, che permette a una persona di dialogare con il sistema, inserendo i fatti del problema e ricevendo le risposte.

3.1 STRUMENTI UTILIZZATI

Per implementare una base di conoscenza in logica è stato utilizzato il linguaggio *Prolog* con l'ausilio della libreria *pyswip*.

La componente di ragionamento del sistema esperto è stata affidata al motore inferenziale *SWI-Prolog*, un ambiente di programmazione che interpreta le regole definite nella Knowledge Base e le applica ai fatti specifici del caso per giungere alla determinazione della personalità.

3.2 KNOWLEDGE BASE

Per il progetto preso in considerazione è stata creata una Knowledge Base nel file “*KB-personality.pl*” sulla base del dominio che si desidera rappresentare, ovvero la determinazione dei tratti della personalità umana basata su un modello di comportamento.

Inizialmente sono state definite le due possibili categorie di output che il sistema è in grado di generare: **tipo_personalita(introverso)**. e **tipo_personalita(estroverso)**.

Successivamente, sono state definite le regole, cioè le proposizioni che saranno considerate vere nell'interpretazione del dominio. Esse rappresentano le verità fondamentali del dominio e costituiscono la base su cui è fondato il ragionamento logico all'interno del sistema.

Per rendere il sistema più accurato, è stato implementato un **meccanismo di valutazione a punteggio**: ad ogni tratto della personalità viene associato un **peso numerico compreso tra 0.1 e 0.9** che rappresenta il grado di appartenenza del tratto a uno dei due profili di personalità sulla base dei valori che il tratto stesso può assumere.

```
% tempo_da_solo
peso(introverso, tempo_da_solo, alto, 0.9).
peso(introverso, tempo_da_solo, medio, 0.5).
peso(introverso, tempo_da_solo, basso, 0.1).

peso(estroverso, tempo_da_solo, alto, 0.1).
peso(estroverso, tempo_da_solo, medio, 0.4).
peso(estroverso, tempo_da_solo, basso, 0.9).
```

Figure 16: Esempio di assegnazione del peso ad un tratto della personalità

Una volta acquisiti i dati mediante l'interfaccia utente, il sistema calcola, per ciascuna tipologia di personalità (*introverso o estroverso*), la media dei pesi associati ai valori forniti.

```
% somma pesi per ogni tipo
punteggio(Persona, Tipo, Totale) :-
    findall(Peso, (ha_tratto(Persona, Tratto, Valore), peso(Tipo, Tratto, Valore, Peso)), ListaPesi), media
    (ListaPesi, Totale).

% media dei pesi
media(Lista, Media) :- sum_list(Lista, Somma), length(Lista, Len), Len > 0, Media is Somma / Len.
```

Figure 17: *Calcolo dei pesi*

Infine, il sistema effettua la classificazione confrontando le due medie calcolate precedentemente per la personalità introversa e per quella estroversa.

```
% ----- CLASSIFICAZIONE -----
classificazione(Persona) :-
    punteggio(Persona, introverso, P1),
    punteggio(Persona, estroverso, P2),
    Diff is abs(P1 - P2),
    (Diff < 0.1 -> Tipo = indefinito;
     P1 > P2 -> Tipo = introverso;
     Tipo = estroverso),
    upcase_atom(Tipo, TipoMaiuscolo),
    format(" \nRISULTATO: la personalita' dell'utente e' -> ~w~n", [TipoMaiuscolo]),
    format(" \nSpiegazione:~n"),
    format(" - Punteggio Introverso: ~2f~n", [P1]),
    format(" - Punteggio Estroverso: ~2f~n", [P2]),
    (Tipo == indefinito -> format(" - La differenza tra i punteggi (~2f) e' troppo bassa o uguale a 0 per una
    classificazione affidabile.~n", [Diff]));
    format(" - La scelta e' stata fatta perche' il punteggio per ~w e' piu' alto di ~2f rispetto all'altro tipo.
    ~n", [TipoMaiuscolo, Diff])).
```

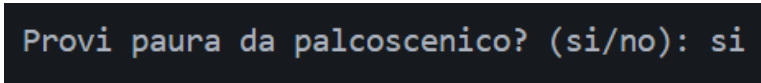
Figure 18: *Classificazione della personalità*

Se la differenza tra le due medie risulta inferiore alla soglia prefissata (0.1), il sistema restituisce *"INDEFINITO"*, ad indicare che i tratti osservati non consentono una classificazione netta. In caso contrario, viene assegnata la personalità corrispondente alla media più elevata, con annessa motivazione della decisione presa dal sistema esperto.

3.3 INTERFACCIA UTENTE

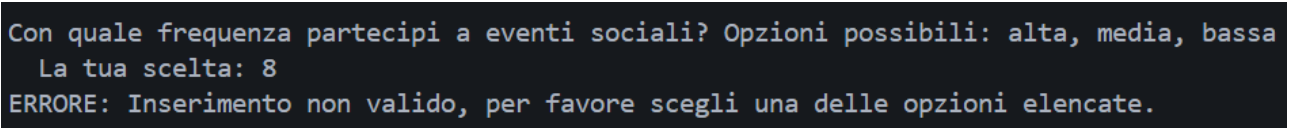
L'interfaccia utente del sistema è stata implementata nel file *“expert_system_personality.py”*. Il suo scopo principale è fare da mediatore tra l'utente finale e il motore inferenziale di Prolog, permettendo un'interazione semplice senza che l'utente debba conoscere la sintassi o la logica della Knowledge Base definita.

Inizialmente viene avviato un dialogo con l'utente in cui il sistema pone delle domande e, attraverso delle funzioni specifiche, raccoglie e valida le risposte fornite dall'utente, continuando a richiederle (in caso di errore) finchè non vengono inserite nel formato corretto.



```
Provi paura da palcoscenico? (si/no): si
```

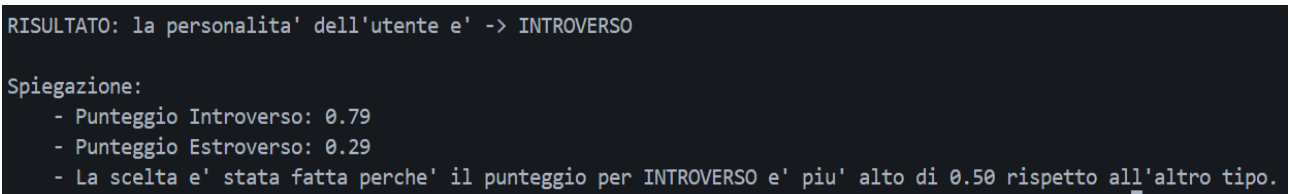
Figure 19: *Esempio inserimento corretto*



```
Con quale frequenza partecipi a eventi sociali? Opzioni possibili: alta, media, bassa  
La tua scelta: 8  
ERRORE: Inserimento non valido, per favore scegli una delle opzioni elencate.
```

Figure 20: *Esempio inserimento errato*

Tutte le risposte fornite dall'utente vengono tradotte e asserite dinamicamente nella Knowledge Base. Infine, il sistema interroga il motore inferenziale per ottenere la previsione finale.



```
RISULTATO: la personalita' dell'utente e' -> INTROVERSO  
Spiegazione:  
- Punteggio Introverso: 0.79  
- Punteggio Estroverso: 0.29  
- La scelta e' stata fatta perche' il punteggio per INTROVERSO e' piu' alto di 0.50 rispetto all'altro tipo.
```

Figure 21: *Esempio risultato finale*

```
-----Sistema Esperto per la Personalità-----

Questo sistema sarà in grado di determinare la tua personalità (introverso oppure estroverso).
Rispondi alle domande per ottenere una previsione.

Provi paura da palcoscenico? (si/no): no

Ti senti svuotato dopo aver socializzato? (si/no): no

Con quale frequenza partecipi a eventi sociali? Opzioni possibili: alta, media, bassa
La tua scelta: media

Con quale frequenza esci? Opzioni possibili: alta, media, bassa
La tua scelta: alta

Con quale frequenza pubblichi post sui social media? Opzioni possibili: alta, media, bassa
La tua scelta: media

Quante ore al giorno trascorri da solo in media? 5

Quante persone ci sono nella tua cerchia di amici più stretta? 7

--- Analisi della personalità in corso... ---

RISULTATO: la personalita' dell'utente e' -> ESTROVERSO

Spiegazione:
- Punteggio Introverso: 0.40
- Punteggio Estroverso: 0.67
- La scelta e' stata fatta perche' il punteggio per ESTROVERSO e' piu' alto di 0.27 rispetto all'altro tipo.
```

Figure 22: *Esempio classificazione*